UNIT IV

BOLTZMANN MACHINES (9 Hrs)

Introduction to Boltzmann Machine – Energy-Based Models – Restricted Boltzmann Machine – Contrastive Divergence – Deep Belief Networks – Deep Boltzmann Machine

The Boltzmann distribution is a probability distribution that gives the probability of a system being in a certain state as a function of that state's energy and the temperature of the system.

It was formulated by Ludwig Boltzmann in 1868 and is also known as the Gibbs distribution.

What are Boltzmann machines used for?

The main aim of a Boltzmann machine is to optimize the solution of a problem. To do this, it optimizes the weights and quantities related to the specific problem that is assigned to it. This technique is employed when the main aim is to create mapping and to learn from the attributes and target variables in the data. If you seek to identify an underlying structure or the pattern within the data, unsupervised learning methods for this model are regarded to be more useful. Some of the most widely used unsupervised learning methods are clustering, dimensionality reduction, anomaly detection and creating generative models.

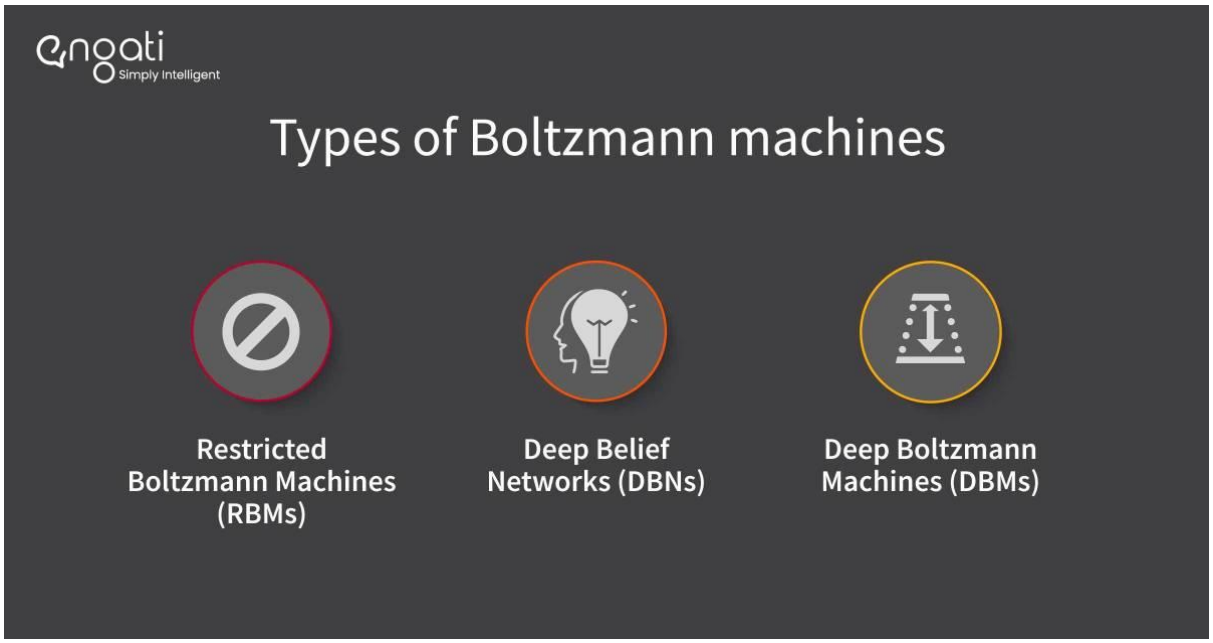
All of these techniques have a different objective of detecting patterns like identifying latent grouping, finding irregularities in the data, or even generating new samples from the data that is available. You can even stack these networks in layers to build deep neural networks that capture highly complicated statistics. Restricted Boltzmann machines are widely used in the domain of imaging and image processing as well because they have the ability to model continuous data that are common to natural images. They are even used to solve complicated quantum mechanical many-particle problems or classical statistical physics problems like the Ising and Potts classes of models.

How does a Boltzmann machine work?

Boltzmann machines are non-deterministic (stochastic) generative Deep Learning models that only have two kinds of nodes - hidden and visible nodes. They don’t have any output nodes, and that’s what gives them the non-deterministic feature. They learn patterns without the typical 1 or 0 type output through which patterns are learned and optimized using Stochastic Gradient Descent.

A major difference is that unlike other traditional networks (A/C/R) which don’t have any connections between the input nodes, Boltzmann Machines have connections among the input nodes. Every node is connected to all other nodes irrespective of whether they are input or hidden nodes. This enables them to share information among themselves and self-generate subsequent data. You’d only measure what’s on the visible nodes and not what’s on the hidden nodes. After the input is provided, the Boltzmann machines are able to capture all the parameters, patterns and correlations among the data. It is because of this that they are known as deep generative models and they fall into the class of Unsupervised Deep Learning.

What are the types of Boltzmann machines?



There are three types of Boltzmann machines. These are:

- Restricted Boltzmann Machines (RBMs)
- Deep Belief Networks (DBNs)
- Deep Boltzmann Machines (DBMs)

1. Restricted Boltzmann Machines (RBMs)

While in a full Boltzmann machine all the nodes are connected to each other and the connections grow exponentially, an RBM has certain restrictions with respect to node connections.

In a Restricted Boltzmann Machine, hidden nodes cannot be connected to each other while visible nodes are connected to each other.

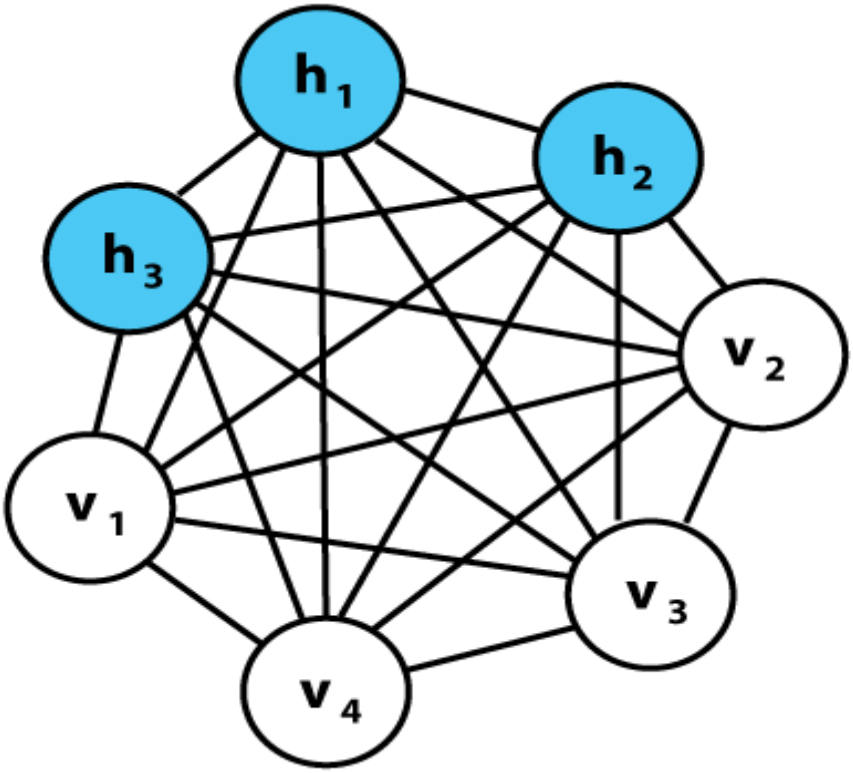
2. Deep Belief Networks (DBNs)

In a Deep Belief Network, you could say that multiple Restricted Boltzmann Machines are stacked, such that the outputs of the first RBM are the inputs of the subsequent RBM. The connections within individual layers are undirected, while the connections between layers are directed. However, there is an exception here. The connection between the top two layers is undirected.

A deep belief network can either be trained using a Greedy Layer-wise Training Algorithm or a Wake-Sleep Algorithm.

3. Deep Boltzmann Machines (DBMs)

Deep Boltzmann Machines are very similar to Deep Belief Networks. The difference between these two types of Boltzmann machines is that while connections between layers in DBNs are directed, in DBMs, the connections within layers, as well as the connections between the layers, are all undirected.



Energy in Boltzmann machines:

In Boltzmann machines, the concept of energy plays a crucial role in defining the model's behavior and learning process. The energy of a Boltzmann machine configuration is determined by the states of its units and the parameters (weights and biases) of the model. Here's a more detailed explanation of energy in Boltzmann machines:

1. Energy Function: The energy E(v, h) of a Boltzmann machine configuration is defined as a function of the visible units v and the hidden units h. Mathematically, it can be expressed as: E(v, h) = - Σα¡vi - j bjhj - Σε Σ; ViWijhj

Where:

Vi and hij are the binary states of visible and hidden units, respectively.

• a_i and bj are the biases associated with visible and hidden units, respectively.

Wij represents the weights of connections between visible unit i and hidden unit j.

2. Role of Energy: In Boltzmann machines, configurations with lower energy are more probable than

those with higher energy. This concept is inspired by statistical physics, specifically the Boltzmann distribution, where states with lower energy are more likely in thermodynamic systems.

3. Probability Distribution: The probability of a configuration (v, h) occurring in a Boltzmann machine is proportional to the negative exponent of its energy divided by a normalization factor (partition function): E(v,h)

P(v,h) = Where Z is the partition function, which ensures that the probabilities sum up to 1 over all possible configurations. Z

4. Sampling: Boltzmann machines use the energy function to perform Gibbs sampling, which is a Markov Chain Monte Carlo (MCMC) technique. During Gibbs sampling, the states of units are iteratively updated probabilistically based the current states of other units and the energy function.

5. Learning: The parameters of the Boltzmann machine, including weights Wij and biases a_i and bj, are learned from data. Techniques like Contrastive Divergence (CD) approximate the gradient of the log-likelihood of the model parameters by adjusting them to reduce the difference in energy between observed data and samples generated by the model.

Overall, energy serves as a fundamental concept in Boltzmann machines, influencing both the probability distribution over configurations and the learning process to adjust model parameters.

Restricted Boltzmann Machine:

Introduction :

Restricted Boltzmann Machine (RBM) is a type of artificial neural network that is used for unsupervised learning. It is a type of generative model that is capable of learning a probability distribution over a set of input data.

RBM was introduced in the mid-2000s by Hinton and Salakhutdinov as a way to address the problem of unsupervised learning. It is a type of neural network that consists of two layers of neurons – a visible layer and a hidden layer. The visible layer represents the input data, while the hidden layer represents a set of features that are learned by the network.

The RBM is called “restricted” because the connections between the neurons in the same layer are not allowed. In other words, each neuron in the visible layer is only connected to neurons in the hidden layer, and vice versa. This allows the RBM to learn a compressed representation of the input data by reducing the dimensionality of the input.

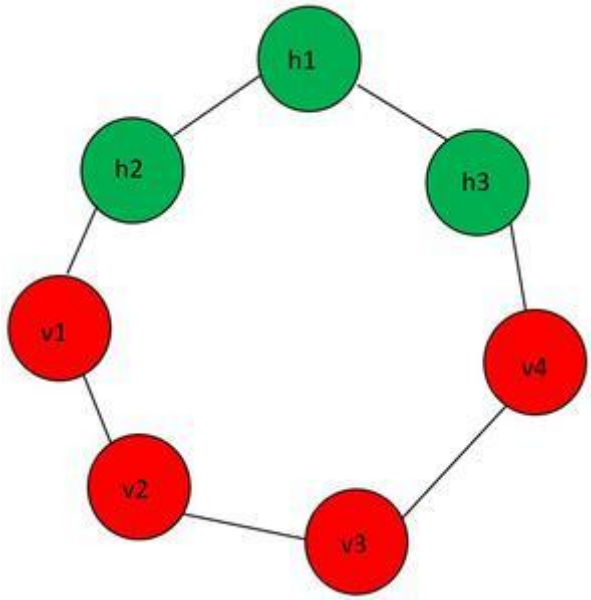
The RBM is trained using a process called contrastive divergence, which is a variant of the stochastic gradient descent algorithm. During training, the network adjusts the weights of the connections between the neurons in order to maximize the likelihood of the training data. Once the RBM is trained, it can be used to generate new samples from the learned probability distribution.

RBM has found applications in a wide range of fields, including computer vision, natural language processing, and speech recognition. It has also been used in combination with other neural network architectures, such as deep belief networks and deep neural networks, to improve their performance.

What are Boltzmann Machines?

It is a network of neurons in which all the neurons are connected to each other. In this machine, there are two layers named visible layer or input layer and hidden layer. The visible layer is denoted as **v** and the hidden layer is denoted as the **h.** In Boltzmann machine, there is no output layer. Boltzmann machines are random and generative neural networks capable of learning internal representations and are able to represent and (given enough time) solve tough combinatoric problems.

The Boltzmann distribution (also known as **Gibbs Distribution**) which is an integral part of Statistical Mechanics and also explain the impact of parameters like Entropy and Temperature on the Quantum States in Thermodynamics. Due to this, it is also known as **Energy-Based Models (EBM)**. It was invented in 1985 by Geoffrey Hinton, then a Professor at Carnegie Mellon University, and Terry Sejnowski, then a Professor at Johns Hopkins University



What are Restricted Boltzmann Machines (RBM)?

A restricted term refers to that we are not allowed to connect the same type layer to each other. In other words, the two neurons of the input layer or hidden layer can't connect to each other. Although the hidden layer and visible layer can be connected to each other.

As in this machine, there is no output layer so the question arises how we are going to identify, adjust the weights and how to measure the that our prediction is accurate or not. All the questions have one answer, that is Restricted Boltzmann Machine.

The RBM algorithm was proposed by Geoffrey Hinton (2007), which learns probability distribution over its sample training data inputs. It has seen wide applications in different areas of supervised/unsupervised machine learning such as feature learning, dimensionality reduction, classification, collaborative filtering, and topic modeling.

Consider the example movie rating discussed in the recommender system section.

Movies like Avengers, Avatar, and Interstellar have strong associations with the latest fantasy and science fiction factor. Based on the user rating RBM will discover latent factors that can explain the activation of movie choices. In short, RBM describes variability among correlated variables of input dataset in terms of a potentially lower number of unobserved variables.

The energy function is given by

$$E(v, h) = -a^T v - b^T h - v^T W h$$

Applications of Restricted Boltzmann Machine

Restricted Boltzmann Machines (RBMs) have found numerous applications in various fields, some of which are:

- **Collaborative filtering:** RBMs are widely used in collaborative filtering for recommender systems. They learn to predict user preferences based on their past behavior and recommend items that are likely to be of interest to the user.
- **Image and video processing:** RBMs can be used for image and video processing tasks such as object recognition, image denoising, and image reconstruction. They can also be used for tasks such as video segmentation and tracking.
- **Natural language processing:** RBMs can be used for natural language processing tasks such as language modeling, text classification, and sentiment analysis. They can also be used for tasks such as speech recognition and speech synthesis.
- **Bioinformatics:** RBMs have found applications in bioinformatics for tasks such as protein structure prediction, gene expression analysis, and drug discovery.
- **Financial modeling:** RBMs can be used for financial modeling tasks such as predicting stock prices, risk analysis, and portfolio optimization.
- **Anomaly detection:** RBMs can be used for anomaly detection tasks such as fraud detection in financial transactions, network intrusion detection, and medical diagnosis.
- It is used in Filtering.
- It is used in Feature Learning.
- It is used in Classification.
- It is used in Risk Detection.
- It is used in Business and Economic analysis.

How do Restricted Boltzmann Machines work?

In RBM there are two phases through which the entire RBM works:

1st Phase: In this phase, we take the input layer and using the concept of weights and biased we are going to activate the hidden layer. This process is said to be Feed Forward Pass. In Feed Forward Pass we are identifying the positive association and negative association.

Feed Forward Equation:

- **Positive Association** — When the association between the visible unit and the hidden unit is positive.
- **Negative Association** — When the association between the visible unit and the hidden unit is negative.

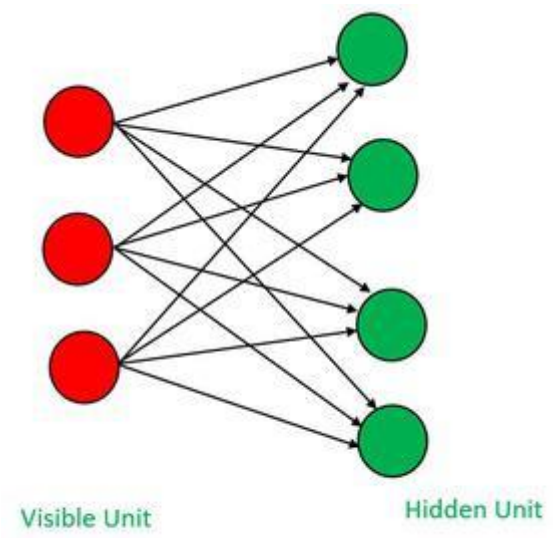
2nd Phase: As we don't have any output layer. Instead of calculating the output layer, we are reconstructing the input layer through the activated hidden state. This process is said to be Feed Backward Pass. We are just backtracking the input layer through the activated hidden neurons. After performing this we have reconstructed Input through the activated hidden state. So, we can calculate the error and adjust weight in this way:

Feed Backward Equation:

- **Error =** Reconstructed Input Layer-Actual Input layer
- **Adjust Weight =** Input*error*learning rate (0.1)

After doing all the steps we get the pattern that is responsible to activate the hidden neurons. To understand how it works:

Let us consider an example in which we have some assumption that V1 visible unit activates the h1 and h2 hidden unit and V2 visible unit activates the h2 and h3 hidden. Now when any new visible unit let V5 has come into the machine and it also activates the h1 and h2 unit. So, we can back trace the hidden units easily and also identify that the characteristics of the new V5 neuron is matching with that of V1. This is because V1 also activated the same hidden unit earlier.



Restricted Boltzmann Machines

Types of RBM :
There are mainly two types of Restricted Boltzmann Machine (RBM) based on the types of variables they use:

1. **Binary RBM:** In a binary RBM, the input and hidden units are binary variables. Binary RBMs are often used in modeling binary data such as images or text.
2. **Gaussian RBM:** In a Gaussian RBM, the input and hidden units are continuous variables that follow a Gaussian distribution. Gaussian RBMs are often used in modeling continuous data such as audio signals or sensor data.

Apart from these two types, there are also variations of RBMs such as:

1. **Deep Belief Network (DBN):** A DBN is a type of generative model that consists of multiple layers of RBMs. DBNs are often used in modeling high-dimensional data such as images or videos.
2. **Convolutional RBM (CRBM):** A CRBM is a type of RBM that is designed specifically for processing images or other grid-like structures. In a CRBM, the connections between the input and hidden units are local and shared, which makes it possible to capture spatial relationships between the input units.
3. **Temporal RBM (TRBM):** A TRBM is a type of RBM that is designed for processing temporal data such as time series or video frames. In a TRBM, the hidden units are connected across time steps, which allows the network to model temporal dependencies in the data.

Here's a complete roadmap for you to become a developer: **Learn DSA -> Master Frontend/Backend/Full Stack -> Build Projects -> Keep Applying to Jobs** And why go anywhere else when our DSA to Development: Coding Guide helps you do this in a single

program!
Apply now to our DSA to Development Program and our counsellors will connect with you for further guidance & support.

What is Contrastive Divergence?

Contrastive Divergence (CD) is a learning algorithm used primarily for training energy-based models such as Restricted Boltzmann Machines (RBMs) and certain types of Markov Random Fields. Developed by Geoffrey Hinton and his colleagues, CD is an approximation technique that speeds up the training of these models, which are otherwise computationally expensive due to the intractability of their partition functions.

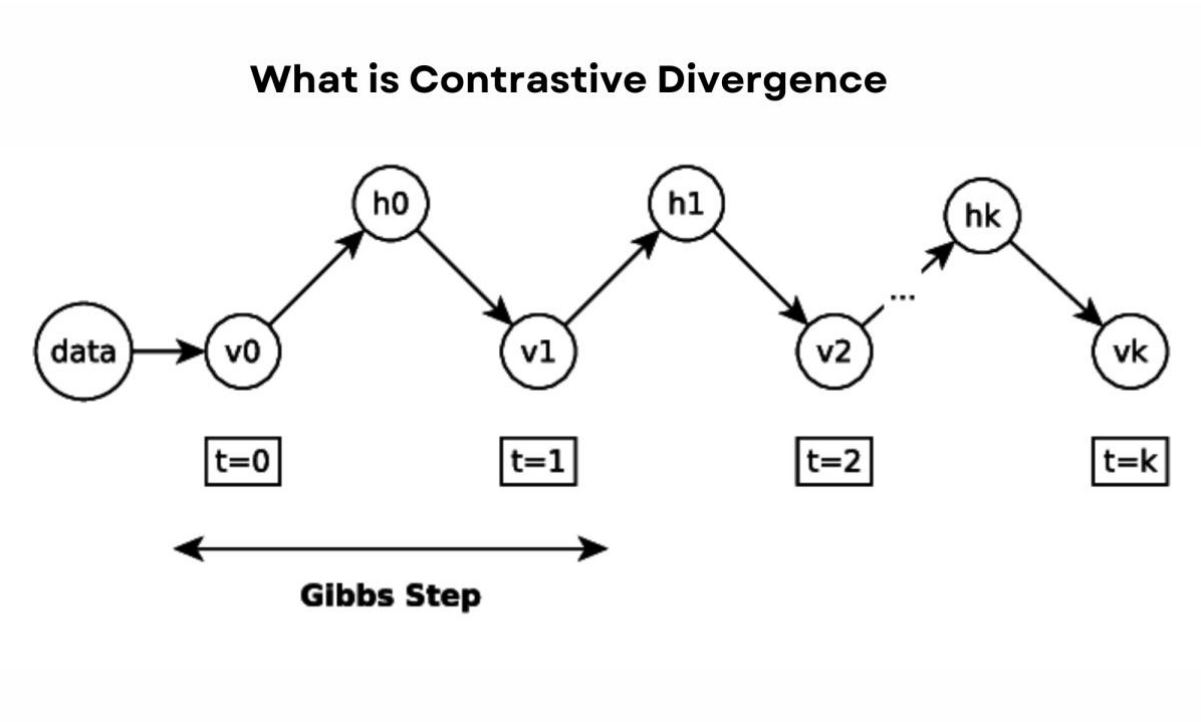
Understanding Energy-Based Models

Before diving into Contrastive Divergence, it's important to understand the context in which it is used. Energy-based models (EBMs) like RBMs are a class of probabilistic graphical models that assign a scalar energy to each configuration of the variables they represent. The probability of a configuration is then defined in terms of this energy, with lower-energy configurations being more probable. However, computing the overall probability distribution requires summing over all possible configurations, which is often infeasible for large systems due to the exponential number of terms involved.

The Role of Contrastive Divergence

Contrastive Divergence addresses the computational challenge posed by the partition function in EBMs. The key insight behind CD is that it's possible to train these models without having to calculate the full partition function. Instead, CD focuses on adjusting the model parameters so that the probability of the observed data increases while the probability of samples generated by the model decreases.

To achieve this, CD performs a Gibbs sampling procedure starting from the training data to produce samples that the model believes are likely. It then uses these samples to estimate the gradient of the log-likelihood of the training data with respect to the model parameters. This gradient is used to update the parameters in a direction that improves the model's representation of the data.



Contrastive Divergence Algorithm

The CD algorithm can be summarized in the following steps:

1. Start with a training example and compute the probabilities of the hidden units.
2. Sample a hidden configuration based on these probabilities, yielding a "reconstruction" of the visible units.
3. Update the model parameters based on the difference between the outer products of the original training example and the reconstructed sample.

The algorithm typically only runs the Gibbs sampler for a few steps (often just one), which is why the method is called Contrastive Divergence, as it contrasts the data-driven and model-driven updates. The "divergence" part of the name refers to the fact that the algorithm minimizes a divergence measure between the data distribution and the model distribution.

Advantages and Limitations

The primary advantage of Contrastive Divergence is its efficiency. By avoiding the direct computation of the partition function, CD allows for much faster training of EBMs, making them practical for real-world applications. However, CD is an approximation method, and the quality of the approximation depends on the number of Gibbs sampling steps taken. In practice, CD can lead to biased estimates of the model parameters, and the resulting models may not perfectly represent the probability distribution of the data.

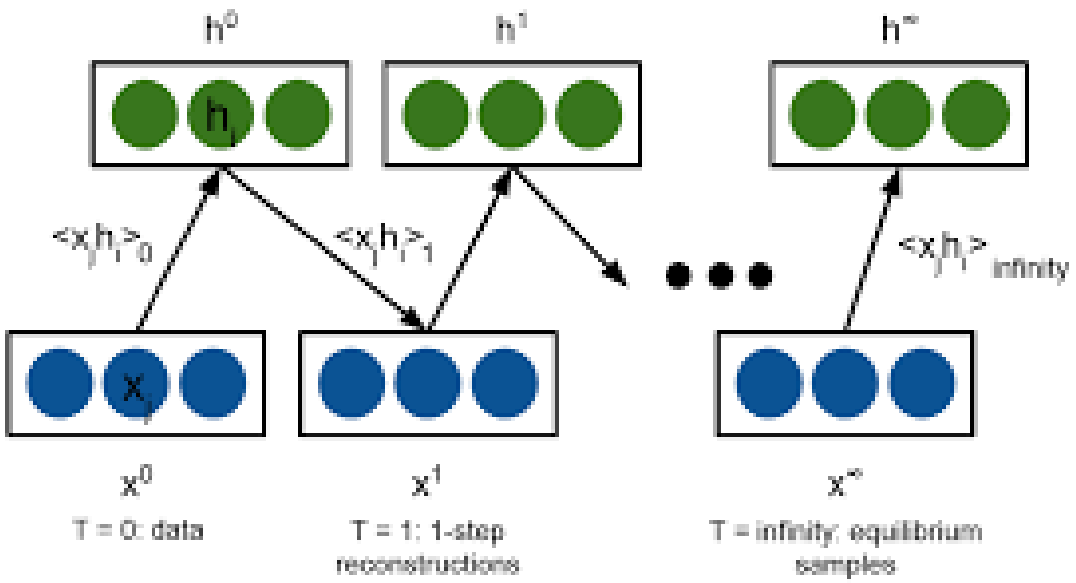
Applications

Contrastive Divergence has been used successfully in various domains, particularly in unsupervised learning tasks where the goal is to learn a good representation of the input data. Applications include dimensionality reduction, feature learning, and as a pre-training step for

deeper neural networks. RBMs trained with CD have been used in collaborative filtering, classification tasks, and even in the initial layers of deep learning architectures for image and speech recognition.

Conclusion

Contrastive Divergence is a powerful algorithm that has facilitated the training of complex probabilistic models like RBMs. By providing a practical way to train these models, CD has contributed to the advancement of unsupervised learning and deep learning. While it is not without its limitations, the efficiency gains offered by CD make it an important tool in the machine learning toolbox, particularly when dealing with large datasets and complex models.



Deep Belief Network?

Deep belief networks (DBNs) are a type of deep learning algorithm that addresses the problems associated with classic neural networks. They do this by using layers of stochastic latent variables, which make up the network. These binary latent variables, or feature detectors and hidden units, are binary variables, and they are known as stochastic because they can take on any value within a specific range with some probability.

The top two layers in DBNs have no direction, but the layers above them have directed links to lower layers. DBNs differ from traditional neural networks because they can be generative and discriminative models. For example, you can only train a conventional neural network to classify images.

DBNs also differ from other deep learning algorithms like restricted Boltzmann machines (RBMs) or autoencoders because they don't work with raw inputs like RBMs. Instead, they rely on an input layer with one neuron per input vector and then proceed through many layers until reaching a final layer where outputs are generated using probabilities derived from previous layers' activations!

How Did Deep Belief Neural Networks Evolve?

Perceptrons, the first generation of neural networks, are incredibly powerful. You can use them to identify an object in an image or tell you how much you like a particular food based on your reaction. But they're limited. They typically only consider one piece of information at a time and can't believe the context of what's happening around them.

To address these problems, we need to get creative! And that's where second-generation neural networks come in. Backpropagation is a method that compares the received output with the desired outcome and reduces the error value until it reaches zero, which means that each perceptron will eventually get its optimal state.

The next step is directed acyclic graphs (DAGs), also known as belief networks, which aid in solving inference and learning problems. Giving us more power over our data than ever before!

Finally, we can use deep belief networks (DBNs) to help construct fair values that we can store in leaf nodes, meaning that no matter what happens along the way, we'll always have an accurate answer right at our fingertips!

The Architecture of DBN

In the DBN, we have a hierarchy of layers. The top two layers are the associative memory, and the bottom layer is the visible units. The arrows pointing towards the layer closest to the data point to relationships between all lower layers.

Directed acyclic connections in the lower layers translate associative memory to observable variables.

The lowest layer of visible units receives input data as binary or actual data. Like RBM, there are no intralayer connections in DBN. The hidden units represent features that encapsulate the data’s correlations.

A matrix of proportional weights W connects two layers. We’ll link every unit in each layer to every other unit in the layer above it.

How Does DBN work?

Getting from pixels to property layers is not a straightforward process.

First, we train a property layer that can directly gain pixel input signals. Then we learn the features of the preliminarily attained features by treating the values of this subcaste as pixels. The lower bound on the log-liability of the training data set improves every time a fresh subcaste of parcels or features that we add to the network.

The deep belief network's operational pipeline is as follows:

- First, we run numerous steps of Gibbs sampling in the top two hidden layers. The top two hidden layers define the RBM. Thus, this stage effectively extracts a sample from it.
- Then generate a sample from the visible units using a single pass of ancestral sampling through the rest of the model.
- Finally, we'll use a single bottom-up pass to infer the values of the latent variables in each layer. In the bottom layer, greedy pretraining begins with an observed data vector. It then oppositely fine-tunes the generative weights.

Creating a Deep Belief Network

DBNs are made up of multiple Restricted Boltzmann Machines (RBMs), just like regular Boltzmann Machines, except they're not fully connected. That's why we call them "restricted."

We use the fully unsupervised form of DBNs to initialize Deep Neural Networks, whereas we use the classification form of DBNs as classifiers on their own.

It's pretty simple: each record type contains the RBMs that make up the network's layers, as well as a vector indicating layer size and- in the case of classification DBNs- number of classes in representative data set.

Learning a Deep Belief Network

RBMs are the building blocks of deep learning models and are also why they're so easy to train.

RBM training is shorter than DBN training because RBMs are unsupervised. You don't need to feed them labeled data. You train them using your data and let them figure out what's happening.

The RBM is a deep learning model used to implement unsupervised learning, while the DBN model is a type of neural network. The RBM has fewer parameters than the DBN and can be trained faster, but it also can't handle missing values. The DBN can be prepared with missing data, but its training is more complex and requires more time.

Applications

Deep Belief Networks are a more computationally efficient version of feedforward neural networks and can be used for image recognition, video sequences, motion capture data, speech recognition, and more.

1. Are deep belief networks still used?

Deep belief networks (DBNs), which were popular in the early days of deep learning, are less widely used than other algorithms for unsupervised and generative learning. However, they are still crucial to the history of deep learning.

2. What is the difference between deep belief and deep neural networks?

Deep belief networks differ from deep neural networks in that they make connections between layers that are undirected (not pre-determined), thus varying in topology by definition.

3. What type of algorithms are DBNs?

Greedy learning algorithms are used to train deep belief networks. In the greedy approach, the algorithm adds units in top-down layers and learns generative weights that minimize the error on training examples. Gibbs sampling is used to understand the top two hidden layers.

4. Is the deep belief network supervised or unsupervised?

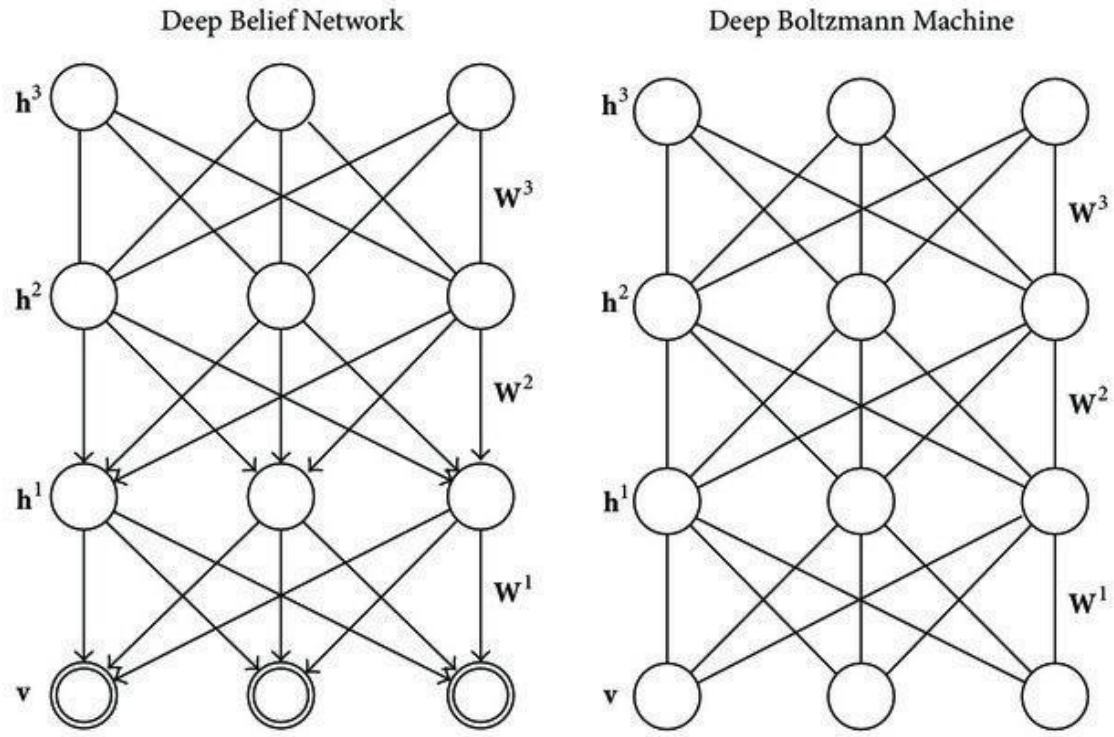
Deep Belief Networks (DBN) is an unsupervised learning algorithm consisting of two different types of neural networks – Belief Networks and Restricted Boltzmann Machines. In contrast to perceptron and backpropagation neural networks, DBN is also a multi-layer belief network.

5. Who invented the deep belief network?

Deep-Learning networks like the Deep Belief Network (DBN), which Geoffrey Hinton created in 2006, are composed of stacked layers of Restricted Boltzmann Machines (RBMs).

6. What is a deep belief network used for?

Deep Belief Networks (DBNs) have been used to address the problems associated with classic neural networks, such as slow learning, becoming stuck in local minima owing to poor parameter selection, and requiring many training datasets.



What are Deep Boltzmann Machines (DBMs)?

Deep Boltzmann Machines (DBMs) are a kind of artificial neural network that belongs to the family of generative models. They are designed to discover intricate structures within large datasets by learning to recreate the input data they're given. Think of a DBM as an artist who, after studying a collection of paintings, learns to create new artworks that could belong to the same collection. Similarly, a DBM analyzes data and learns how to produce new examples that are similar to the original data.

DBMs consist of multiple layers of hidden units, which are like the neurons in our brains. These units work together to capture the probabilities of various patterns within the data. Unlike some other neural networks, all units in a DBM are connected across layers, but not within the same layer, which allows them to create a web of relationships between different features in the data. This structure helps DBMs to be good at understanding complex data like images, text, or sound.

The 'deep' in the Deep Boltzmann Machine refers to the multiple layers in the network, which allow it to build a deep understanding of the data. Each layer captures increasingly abstract

representations of the data. The first layer might detect edges in an image, the second layer might detect shapes, and the third layer might detect whole objects like cars or trees.

Pause

Unmute

×

How Deep Boltzmann Machines Work?

Deep Boltzmann Machines work by first learning about the data in an unsupervised way, which means they look for patterns without being told what to look for. They do this using a process that involves adjusting the connections between units based on the data they see. This process is similar to tuning a radio to get a clear signal; the DBM 'tunes' itself to resonate with the structure of the data.

When a DBM is given a set of data, it uses a stochastic, or random, process to decide whether a hidden unit should be turned on or off. This decision is based on the input data and the current state of other units in the network. By doing this repeatedly, the DBM learns the probability distribution of the data—basically, it gets an understanding of which patterns are likely and which are not.

After the learning phase, you can use a DBM to generate new data. When generating new data, the DBM starts with a random pattern and refines it step by step, each time updating the pattern to be more like the patterns it learned during training.

Concepts Related to Deep Boltzmann Machines (DBMs)

Several key concepts underpin Deep Boltzmann Machines:

- **Energy-Based Models:** DBMs are energy-based models, which means they assign an 'energy' level to each possible state of the network. States that are more likely have lower energy. The network learns by finding states that minimize this energy.
- **Stochastic Neurons:** Neurons in a DBM are stochastic. Unlike in other types of neural networks, where neurons output a deterministic value based on their input, DBM neurons make random decisions about whether to activate.
- **Unsupervised Learning:** DBMs learn without labels. They look at the data and try to understand the underlying structure without any guidance on what features are important.
- **Pre-training:** DBMs often go through a pre-training phase where they learn one layer at a time. This step-by-step learning helps in stabilizing the learning process before fine-tuning the entire network together.
- **Fine-Tuning**: After pre-training, DBMs are fine-tuned, which means they adjust all their parameters at once to better model the data.

Mathematical concepts

Deep Boltzmann Machines (DBMs) are grounded in some fascinating mathematical concepts, with probability playing a starring role. At the heart of DBMs is the idea of modelling the data using a probability distribution, which is mathematically defined by an energy function. The energy function (h)E(v,h) captures the relationship between visible units v (data) and hidden units h (features).

The probability of a certain state (a combination of visible and hidden units) is given by the Boltzmann distribution:

where Z is the partition function, a normalization factor that ensures all probabilities sum up to one. It's calculated as the sum of e^{-E(v,h)} over all possible states.

Learning in DBMs involves finding the weights that minimize the energy function, which in turn maximizes the probability of the observed data. This is typically done using a learning algorithm like Contrastive Divergence (CD) or Stochastic Gradient Descent (SGD), which adjust the weights to lower the energy of data states and increase their probability.

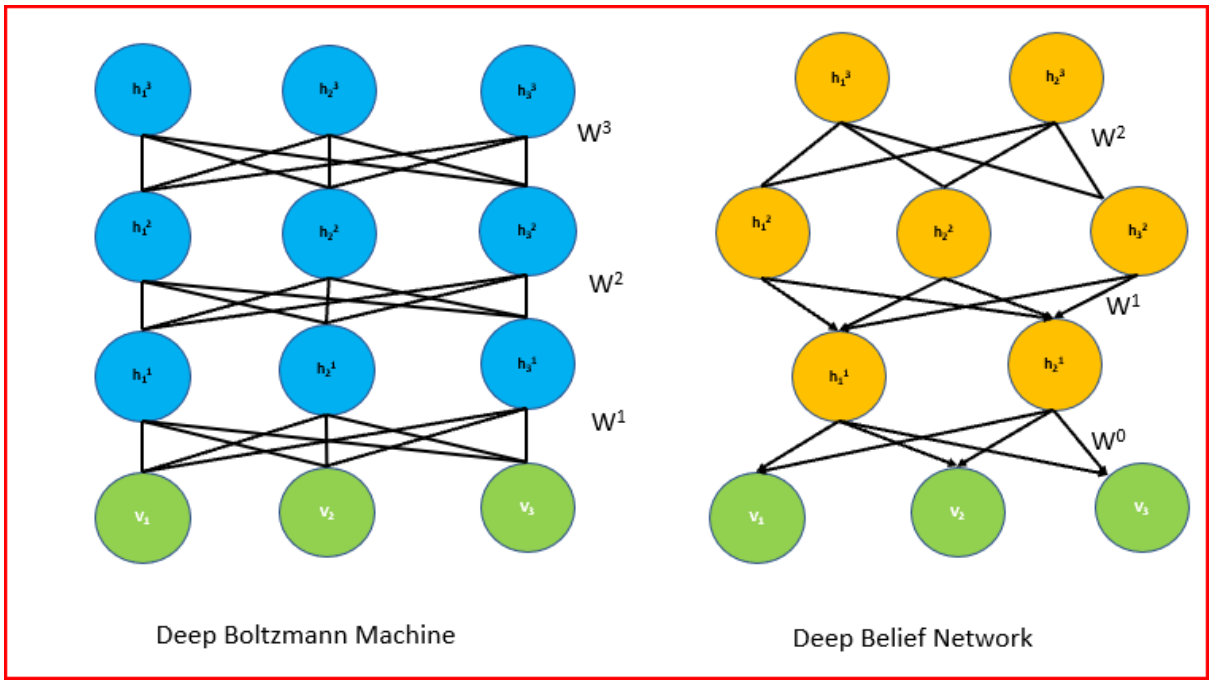
During this process, a DBM learns the weights through repeated sampling. The sampling uses a Markov Chain Monte Carlo (MCMC) method, allowing the model to explore different states based on their probabilities.

In essence, DBMs use the language of statistical mechanics to model data in a probabilistic framework, balancing complex interactions between layers to capture the essence of the data in a way that can be intuitively visualized as a landscape of hills and valleys, where the data points naturally settle into the lowest points, or the states of lowest energy.

Implementation of Deep Boltzmann Machines (DBMs)

This code defines a Deep Boltzmann Machine (DBM) using Python, a type of generative neural network useful for unsupervised learning tasks.

Class RBM:

Represents a Restricted Boltzmann Machine, a fundamental building block of a DBM. An RBM has visible and hidden units and learns a probability distribution over inputs.

- __init_: Initializes weights and biases randomly.
- sample_hidden: Given visible units, it computes the activations of the hidden units.
- sample_visible: Given hidden units, it calculates the activations of the visible units.
- train: Trains the RBM using Contrastive Divergence, adjusting weights and biases to learn the data distribution.